

On this page

*Last updated on Sep 1, 2026*

Best practices for ABAC policies

Consider the following best practices for ABAC policy design and tag governance.

Standardize attributes and naming

Establish a consistent tagging taxonomy before creating policies. Agree on tag key names, allowed values, and naming conventions across teams. A small, well-defined set of tags is easier to manage than a proliferation of ad-hoc tags.

For example, use a single `sensitivity` tag with controlled values (`public`, `internal`, `confidential`, `restricted`) rather than multiple overlapping tags like `is_sensitive`, `data_class`, and `pii_level`.

Control who can set tags

Tagging is a security boundary in ABAC. If a user can change tags on a data asset, they can change which policies apply to it. Wrong or missing tags can leave data unprotected or inaccessible because policies only apply when the right tags are in place.

- Restrict tag creation and modification to authorized data stewards or governance admins. See [Governed tags](#) for how to configure tag permissions.
- Audit tag changes regularly using the [audit log system table](#).

Set fallback rules for unclassified data



Ask Genie

Don't assume that all objects are correctly tagged. Use automation to enforce tagging standards and implement fallback mechanisms for unclassified data:

- Apply a default restrictive tag (like `classification : unverified`) to new objects until a data steward reviews them.
- Create a policy that restricts access to objects with the default tag.

For a detailed example, see [Prevent access until sensitive columns are tagged](#).

Define policies at the highest applicable scope

Attach policies at the catalog or schema level when possible. Table-level policies are rare and should be the exception.

Catalog-scoped policies evaluate against all tables in the catalog, and schema-scoped policies evaluate against all tables in the schema. When you add new tables, existing policies apply as long as their tags match the policy's conditions.

Avoid policy sprawl

ABAC is designed to reduce the number of access control rules, not increase them. If teams create too many tags and policies, the result is hard to manage and audit.

- Analyze your governance requirements before creating policies.
- Start with a small number of broad policies, such as PII masking across a catalog or regional row filtering.
- Avoid creating a separate policy for every edge case.
- Review policies periodically and consolidate overlapping ones.

Large numbers of policies and complex conditions can slow authorization checks. See [Performance considerations](#) for details.

Audit direct grants and ABAC GRANT policies together

TIP

A user's effective privileges on a securable object are the union of both direct grants and ABAC GRANT policies. When reviewing access, check both direct grants and ABAC GRANT policies. Reviewing only one can give you an incomplete view of the user's effective privileges. See [ABAC GRANT policies](#).

Prefer TO/EXCEPT for principal targeting

For row filter and column mask policies, use the policy's `TO` and `EXCEPT` clauses to define which users and groups the policy applies to. This keeps UDF logic simple. The `EXCEPT` clause excludes specific users from the policy entirely so they are not subject to any filtering or masking. When complex conditional logic is required, identity functions like `is_account_group_member()` inside UDFs remain a valid option.

For GRANT policies, `TO` and `EXCEPT` are the only mechanisms for targeting principals because GRANT policies do not use UDFs.

You can also target users by an identity attribute such as `department`. When the attribute already exists in your identity provider, one column mask policy covers every value of the attribute, so you avoid creating a separate group and policy for each one. Make sure users without a value for the attribute do not get access by default. See [Mask a column based on attributes of the querying user](#).

For details, see [Approach for targeting principals](#).

Plan for dynamic policy evaluation

ABAC policies are dynamic. Unlike table-level row filters and column masks, which are directly visible on the table definition, ABAC policies evaluate at query time based on the  **Ask Genie**

user's identity and group memberships, and the tags on the data object in the policy scope. This can make it harder for data consumers and table owners to understand which access rules apply to a given table.

- Use `SHOW EFFECTIVE POLICIES` to determine what applies to a specific table.
- Document your tagging taxonomy, policies, and group management approach so that teams can understand the governance model without inspecting each policy individually.
- If transparency is critical for a specific table, consider using [table-level row filters and column masks](#) for that isolated case instead. Make sure to address possible conflicts first.

Learn more

Topic	Description	
Performance considerations	How ABAC policy design affects query performance, and how to optimize and test your policies.	
When to use ABAC vs table-level row filters and column masks	Differences in scope, ownership, and how to choose between the two approaches.	
OpenSharing and ABAC	How to share ABAC-protected tables through OpenSharing, handle recipient-side policies, and set up recipient-local views.	
Use RBAC with ABAC	How ABAC policies behave when users assume a role under RBAC, including identity-function behavior (<code>current_user()</code> , <code>is_member</code> , <code>is_account_group_member</code>) and combined-use patterns.	

On this page

Was this page helpful?

Yes

No

Send feedback

[Privacy Notice](#)

[Terms of Use](#)

[Modern Slavery Statement](#)

[California Privacy](#)

[Your Privacy Choices](#)

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark and the Spark logo are trademarks of the Apache Software Foundation.